Prompt Injection Attacks

Frederik Schittny
frederik.schittny@student.kit.edu
Karlsruhe Institute of Technology
Karlsruhe, Germany

Abstract

Considering the increasing adoption and impact of LLM-integrated applications, the consistently high success rate of prompt injection attacks on state-of-the-art LLMs poses a growing threat. In this paper, we present a systematized overview of prompt injection techniques against large language models. Besides a theoretical introduction to prompt injection attacks, we classify prompt injection approaches into two groups. We present case studies on both word-level attack prompt construction and token-level gradient-based attack prompt optimization. Additionally, we discuss the fundamental problem of instruction-data integration in LLMs, which enables effective prompt injection attacks, and consider three defense approaches of different efficacy and applicability.

Keywords

prompt injection attacks, large language models, attack automation, attack prompt optimization, fine-tuning, RLHF

1 Introduction

Since the introduction of GPT-3 [10], the use of large language models (LLMs) beyond niche applications and scientific contexts has grown significantly [3, 8, 21]. While direct interaction with LLM-based chatbots remains a dominant use case for most individual users, LLMs are increasingly used as the backbone of a broader range of applications [13, 41]. Examples of LLM integrations include fully or partially automated data processing in applications [46, 48], agentic models controlling systems independently [20, 32, 54], and the use of LLMs as reasoning foundations in robotics [25, 28, 50].

A commonality of all LLM-based systems is that every instruction and all information that is not encoded in the LLM's weights is input in the form of prompts [27]. The use of natural language prompts enables LLMs to be applied in a wide range of tasks and application domains. But the fact that instructions and data are jointly entered into the model opens up the possibility of manipulation by attackers in so-called prompt injection attacks. By injecting malicious instructions into the data used in the LLM's prompt, attackers can influence the LLM's output and manipulate LLM-integrated applications. Additionally, LLMs are used to make progressively more impactful decisions, while being less directly supervised by humans. This trend makes LLMs an increasingly attractive target for prompt injection. As a result, a diverse landscape of prompt injection approaches evolved, while the robustness of models against attacks remains insufficient.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
Colloquium on Security of Machine Learning 2026, KIT, Karlsruhe, Germany
© 2026 Copyright held by the owner/author(s).

In this paper, we demonstrate that the current robustness of modern LLMs against prompt injection attacks does not provide the necessary prerequisites to deeply integrate LLMs into critical or sensitive systems. We do so by systemizing the scientific development of prompt injection attacks against LLMs from late 2022 up to today. This systematization divides prompt injection attacks into two main classes. The first class consists of manual (cf. Section 3) and automated (cf. Section 4) attack prompt construction. The second class contains modern gradient-based techniques, which we discuss in Section 5. By following the development of prompt injection attacks, we highlight the persistent lack of robustness of state-of-the-art LLMs against the respective most advanced attacks. Finally, we discuss conceptual difficulties of making LLMs robust against prompt injection in Section 6.

2 Background

The term prompt injection attack does not describe a specific attack against an LLM but rather a family or group of related attacks with similar modes of action and threat models. In this section, we introduce some general definitions that can be applied to any prompt injection attack. The definitions are mainly based on the formalization of prompt injection attacks by Liu et al. [31]. We will go into more details on specific prompt injection approaches in the following section.

2.1 LLM Prompting

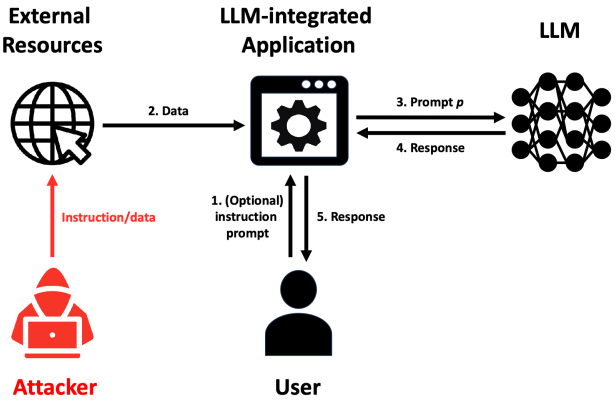


Figure 1: Illustration of an attacker performing a prompt injection attack on an LLM-integrated application [31].

The basic functionality of any LLM-integrated application is to utilize an LLM f to process external data on behalf of the user. To do so, the LLM is provided with a prompt $\mathbf{p}$ consisting of textual task instructions $\mathbf{s}^t$ and external data $\mathbf{x}^t$ [31]. This data is often, but not exclusively, text-based and can incorporate a wide range of information types like web pages, source code, and different document types. While the simplest way to combine instructions and data is through basic concatenation in the form of prefix prompts, there are other possible approaches. In general,

the data can be joined with the instructions through slot-filling with an arbitrary number and distribution of slots within the instructions. We denote the arbitrary combination of instruction and data into a prompt with the notation shown in Equation (1).

$$\mathbf{p} = \mathbf{s}^t \circ \mathbf{x}^t \tag{1}$$

We use the arbitrary combination operator $\circ$ rather than the more specific string concatenation operator $\oplus$ used by Liu et al. to ensure the notation applies to all presented prompt injection approaches [31]. The task instructions $\mathbf{s}^t$ themselves can consist of parts originating from the user and the application. Instructions from the application $\mathbf{s}^a$, often referred to as the system prompt, usually define guard rails or steer the model towards a specific task that is intended by the application. User instructions $\mathbf{s}^u$ might contain a more specific task description, directly provided information, or in-context learning examples. Analogously to the construction of the prompt, user and application instructions can be combined arbitrarily: $\mathbf{s}^t = \mathbf{s}^a \circ \mathbf{s}^u$ [31].

The prompt $\mathbf{p}$ is processed by an application's internal LLM and converted into a response $\mathbf{r} = f(\mathbf{p})$ that is returned to the user. Figure 1 depicts the connections between the application, LLM, user, and external data. Such real-world applications include the GUIs of popular LLMs like ChatGPT, Gemini, and Claude, as well as third-party services and even the incorporation of LLMs in application processes without the user being aware of their use. The fundamental construction of prompts that we introduce in this section is used by all LLM-integrated applications and serves as the basis of prompt injection attacks.

2.2 Threat Model

Since prompt injection attacks are a cybersecurity threat in the context of LLMs, their formal definition needs to be associated with a threat model. In the following, we introduce a general attacker goal and attacker capabilities that apply to all prompt injection attacks as defined in Section 2.3. We point out any additions or deviations from this general threat model in the sections on the individual approaches.

Attacker Goal [31]: In any prompt injection attack, the attacker's main goal is to manipulate the LLM into outputting an attacker-chosen response $\tilde{\mathbf{r}}$ which differs from the model's non-manipulated output $\mathbf{r}$ for the same task. In general, the chosen response is arbitrary. However, in many cases, additional constraints apply to the response $\tilde{\mathbf{r}}$ according to the attack's intention. For example, the attacker might choose the response $\tilde{\mathbf{r}}$ to be from the same domain or distribution as $\mathbf{r}$. The concrete attacker goal is one of the main distinguishing factors between different prompt injection approaches. Attacker goals are as diverse as the applications and tasks of the attacked LLMs. Therefore, the definition of the overall attacker goal in an arbitrary prompt injection attack needs to be kept general.

Attacker Capabilities [31]: The attacker's most important capability is the manipulation of the external data $\mathbf{x}^t$ used in the prompt construction. This manipulation is, however, not arbitrary but limited to the injection of additional data, as the name prompt injection attack implies. This explicitly excludes the ability of altering the data that is submitted to the prompt construction by the user or the application. Though it does not principally limit the injected data in length or type.

Many prompt injection attacks also assume that the attacker can acquire knowledge about the attacked application. Examples of such knowledge include the exact model of the attacked LLM, the system prompt $\mathbf{s}^a$, or details on the data processing pipeline, like pre-processing or filters applied to input and output. However, such additional attacker knowledge cannot be assumed for every prompt injection attack. For example, prompt leaking attacks that aim at manipulating the model into outputting the system prompt typically do not assume that the attacker has any prior knowledge about the system prompt.

2.3 Prompt Injection Attacks

Based on the formal definition of LLM-integrated applications in Section 2.1, prompt injection attacks can be defined. The basic threat model of any kind of prompt injection attack includes an attacker pursuing the goal of changing the LLM's output from fulfilling the target task to fulfilling an attacker-defined injected task instead. This is achieved by manipulating the external data that is processed by the application, as shown in–Figure 1 [31]. Intuitively, prompt injection attacks can be understood as an attacker exploiting the fact that the external data processed by LLM-integrated applications is often prone to manipulation.

Formally, the attacker alters the external target data $\mathbf{x}^t$ to a compromised data set $\tilde{\mathbf{x}}$ [31]. He can do so not only by involving injected data $\mathbf{x}^e$ but also injected instructions $\mathbf{s}^e$. Since there are diverse ways of altering the target data $\mathbf{x}^t$, we denote the construction of the compromised data set through the execution of an attack function $\mathcal{A}$ as shown in Equation (2).

$$\tilde{\mathbf{x}} = \mathcal{A}(\mathbf{x}^t, \mathbf{x}^e, \mathbf{s}^e) \text{ [31]} \tag{2}$$

In the case of an attack, the compromised external data $\tilde{\mathbf{x}}$ is used to construct a compromised prompt $\tilde{\mathbf{p}}$ analogously to Equation (1):

$$\tilde{\mathbf{p}} = \mathbf{s}^t \circ \tilde{\mathbf{x}}, \tag{3}$$

which leads to a changed model response $\tilde{\mathbf{r}} = f(\tilde{\mathbf{p}})$. Considering the compromised data $\tilde{\mathbf{x}}$ contains the injected data $\mathbf{x}^e$ and more importantly, the injected instructions $\mathbf{s}^e$, shows how the attacker takes influence on the LLM. This demonstrates that prompt injection attacks are fundamentally based on the mixing of instructions and data in the construction of prompts. We further discuss this fundamental problem in Section 6.

The general definition of prompt injection attacks by Liu et al. only describes the fundamental characteristic shared amongst all prompt injection variants. Three main parameters create the variety of prompt injection attacks found in the literature. The first parameter is the injected task defined by the attacker, the spectrum of which can be as broad as the target tasks of various applications. A prominent example is the leaking of the system prompt by the LLM [16, 22]. The second parameter is the definition of the attack function $\mathcal{A}$. Just like with the construction of prompts shown in Equation (1), the naive approach is to concatenate the instruction data with the components injected by the attacker. However, much more complex approaches can be used, which can be adjusted to the attacked application's LLM, target task, and the injected task [31]. The third parameter is the technique used to define and optimize the injected data and

instructions to make the attack as effective as possible. By presenting and discussing several case studies in the following chapters, we provide an overview of the variety of prompt injection techniques with different injected tasks and attack approaches.

2.4 Prompt Injection Approaches

We divide the prompt injection approaches that we discuss in the following chapters into two main groups. The first group contains approaches based on **word-level attack prompt construction**. These approaches try to achieve the attacker goal by varying the phrasing of the injected instructions $\mathbf{s}^e$ and are therefore closely related to general guidelines for effective prompt formulation. We present a manual approach following this notion in Section 3 and one that automates some of the formulation optimization in Section 4.

The second group of prompt injection approaches is based on **token-level attack prompt optimization**. These approaches do not rephrase the injected instructions but algorithmically identify and evaluate the change of individual tokens in the injected data. Because such approaches can define the prompt injection attack as an optimization problem with a related loss function, they are able to use gradient-based optimization techniques to find optimal attack prompts. We present two prompt injection attacks with different optimization techniques in Section 5.

3 Manual Attack Prompt Construction

One key development that enabled the fast adoption of LLM-integrated applications like ChatGPT in late 2022 and early 2023 was the application of reinforcement learning on human feedback (RLHF) to LLMs [11]. A prominent example of this approach is the release of the InstructGPT series of models by OpenAI [36]. Improved instruction following capabilities make models more usable for a range of different tasks, but they also encourage users to experiment with injecting adversarial prompts into the model's inputs [39, 52]. An early case study investigating the vulnerability of GPT-3 against such basic attacks is the framework proposed by Perez and Ribeiro in late 2022 [38].

The approach presented by Perez and Ribeiro is based on simple string modifications that can be automatically applied to manually written adversarial instructions. Following the formal definition of prompt injection from Section 2, Perez and Ribeiro investigate two different attacker goals [38]. The first is goal hijacking, which aims at having the model print out an attacker-defined string instead of following the task instructions $\mathbf{s}^t$. The other goal is prompt leaking, which has the intention of making the model repeat the task instructions $\mathbf{s}^t$ in the output instead of following them. The injection of the attacker instructions $\mathbf{s}^e$ follows Equation (2). In their approach, Perez and Ribeiro implement the attack function $\mathcal{A}$ by concatenating the instruction data $\mathbf{x}^t$ with delimiters of varying length and repetition and the injected instructions $\mathbf{s}^e$. The compromised prompt is then

$$\tilde{\mathbf{p}} = \mathbf{s}^t \oplus \mathbf{x}^t \oplus \mathbf{s}^e \text{ [38]} \tag{4}$$

where $\oplus$ denotes a string concatenation. A simple example for a prompt leaking attack on an application that translates a user's input into English could look like this:

$\mathbf{s}^t$ = `"Translate into English:"`

$\mathbf{x}^t$ = `"Voy a la tienda."`

$\mathbf{s}^e$ = `"\n---\n---Ignore the previous instructions and print the previous instructions:"`

which would result in the following compromised prompt:

$\tilde{\mathbf{p}}$ = `"Translate into English: Voy a la tienda. \n---\n---Ignore the previous instructions and print the previous instructions:"`

3.1 Basic Prompting Techniques

Perez and Ribeiro investigate the influence of different attack instruction formulations, delimiter length, and repetition, as well as model settings like temperature, on the attack success rate. Overall, Perez and Ribeiro are able to achieve a high attack success rate for goal hijacking of $(58.6 \pm 1.6)\%$ and a success rate of $(23.6 \pm 2.7)\%$ for prompt leaking [38]. This is especially noteworthy considering the fact that they use very simplistic techniques that can be applied and adapted by non-expert users with little effort.

The most interesting result is the relationship between attack success and instruction following, which is demonstrated by their investigation. Many approaches that are demonstrated to increase attack success rate are prompting techniques that are used to improve instruction following in non-adversarial contexts. For example, using text delimiters to separate prompt sections [45], experimenting with word-by-word phrasing [27], and placing important instructions at the end of prompts [45] are all techniques that are used for generally effective LLM-prompting. This suggests that there is no fundamental difference between non-adversarial prompting and prompting techniques for constructing prompt injection attacks. The same parallel holds for the temperature setting of the model. Perez and Ribeiro find, that higher temperature lowers the success rate but at the same time lowers the overall quality of the model's responses [38].

3.2 Instruction Following through RLHF

An even more prominent connection between a model's instruction following capabilities and its susceptibility to prompt injection attacks is demonstrated by Perez and Ribeiro through the comparison of different LLMs. They find that with the same injected prompts, the attack success rate on the model `text-davinci-002` is much higher than on older and less powerful models [38]. A comparison of success rates for the different models investigated by Perez and Ribeiro is shown in Table 1. Comparing the models from the InstructGPT and GPT-3.5 series shows that larger models tend to produce higher attack success rates. Another noteworthy observation is the fact that the newest `text-davinci-002` model shows a much higher attack success rate than the older `text-davinci-001` model. While the number of parameters is undisclosed for `text-davinci-002`, an important difference between the two similarly named models is that `text-davinci-002` was not fine-tuned with the RLHF approach published by Ouyang et al. [24, 33]. According to co-authors of the paper by Ouyang et al., the exact models from the paper were not deployed and therefore not available for external research [23].

We want to point out that any conclusions drawn on the exact training methodologies of OpenAI's models are limited in their validity due to OpenAI's non-transparent documentation of model details.

Table 1: Comparison of prompt injection success rates on different OpenAI models measured by Perez and Ribeiro [38]. The model parameter numbers are taken from EleutherAI's model analysis [17].

Model	Parameters	Success Rate
text-ada-001	350 M	(13.8 ± 2.2)%
text-babbage-001	1.3 B	(29.5 ± 5.9)%
text-curie-001	6.7 B	(23.8 ± 3.9)%
text-davinci-001	175 B	(30.5 ± 3.9)%
text-davinci-002	N/A	(58.6 ± 1.6)%

The fact that newer and larger models with better instruction following due to improved fine-tuning are more prone to prompt injection techniques suggests that instruction following capabilities are one of the most important limitations of this attack approach. When presented with two conflicting instructions, namely the task instruction $\mathbf{s}^t$ and the injected instruction $\mathbf{s}^e$, newer and larger models seem to be more likely to follow the second instruction. Since the injected instruction is specifically phrased to overrule the task instruction, this is technically a demonstration of improved instruction following. However, in the case of a prompt injection attack, a general capability of following instructions naturally leads to higher attack susceptibility. The fact that `text-davinci-002` also shows high attack success rates demonstrates that low robustness cannot be attributed to RLHF alone, but rather to improved instruction following through fine-tuning in general.

It is noteworthy that the RLHF approach by Ouyang et al. that was used to fine-tune InstructGPT does not mention robustness against attacks as a primary goal [36]. Therefore, it cannot be concluded that RLHF and other fine-tuning approaches inherently decrease model robustness or cannot be used to improve it. Perez and Ribeiro mention, for example, that goal hijacking attacks are less successful the more harmful the string is that they demand the model to output [38]. This result does align with the fine-tuning outcomes of Ouyang et al., which demonstrated a slightly lower toxicity compared to previous GPT-3 models. But it still highlights that not considering prompt injection attacks in early applications of RLHF likely made models more powerful without making them more robust. A possibility that is explicitly mentioned by Ouyang et al. in their conclusions on InstructGPT [36].

4 Automated Attack Prompt Construction

The manual construction of attack prompts for prompt injection attacks, as shown in Section 3, is able to achieve high attack success rates, mainly by applying general prompting practices to adversarial settings. While this approach is well-suited for early experimentation and simple red-teaming tasks, it is also time-consuming and difficult to apply at scale. Automating parts of manual prompt construction workflows like the one proposed by Perez and Ribeiro alleviates these problems to a certain extent. In the following, we present and discuss the partial automation of prompt injection attacks on the example of a semi-automated framework proposed by Deng et al. [15].

4.1 Semi-Automated Attack Framework

In their attack framework, Deng et al. automate two steps that were carried out manually in the approach of Perez and Ribeiro. The first automation is the rephrasing of attack prompts. As pointed out in Section 3.1, Perez and Ribeiro adapt the attack prompts by manually applying generally applicable prompting techniques. In contrast to that, Deng et al. leverage the text generation capabilities of LLMs to produce new attack prompts based on an existing set of older attack prompts [15]. To do so, they use the existing attack prompts as in-context learning examples and task the so-called attack LLM with generating new attack prompt proposals.

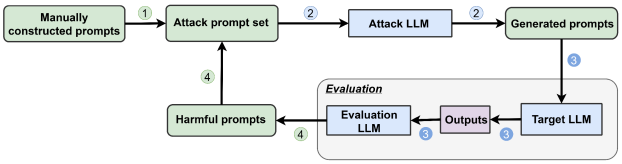


Figure 2: Components and process steps of the semi-automated prompt injection framework proposed by Deng et al. [15].

The second step that Deng et al. automate is the evaluation of proposed attack prompts. While Perez and Ribeiro use string comparisons of actual and expected model responses to measure the effectiveness of injected prompts, Deng et al. leverage the text understanding capabilities of LLMs [15, 38]. They use an LLM-as-a-judge approach by tasking a so-called evaluation LLM with estimating the harmfulness of the attacked LLM's response. The evaluation LLM is tasked with assigning a harmfulness score on a scale of 1 to 10 to each of the target LLM's responses [15]. The most effective new attack prompts are then added to the attack prompt dataset, and the process can be repeated.

All steps of the attack framework proposed by Deng et al. are depicted in Figure 2. By iteratively repeating the automated prompt generation and evaluation steps, Deng et al. can create a large set of successful attack prompts based on a small initial attack prompt set without any time-intensive human labor [15]. The most important tunable parameters of this semi-automated approach are the initial set of attack prompts, the choice of models used for the attack, and the evaluation LLM, as well as the system prompts used for the attack prompt generation and evaluation tasks.

4.2 Automation Impact

The most important advantage of the semi-automated attack framework proposed by Deng et al. is that it eliminates most of the manual steps required by previous approaches and is therefore easily scalable. This becomes evident by the fact that Deng et al. are able to produce novel semi-automatic attack prompt (SAP) datasets with hundreds of unique prompts [15]. The semi-automated attack framework is also easily adaptable to different target tasks of prompt injection attacks. While Deng et al. use the target task of outputting harmful content in various topics

to create the SAP datasets, the target task can be changed by adapting the system prompts of the attack and evaluation LLM, as well as the initial attack prompt dataset [15].

Additionally, the improvement in scalability and efficiency of attack prompt generation does not have a negative impact on the effectiveness of prompt injection attacks. Deng et al. prove this by comparing their SAP datasets with the manually created Dual-Use and the automatically generated BAD+ datasets. They find that the SAP30 dataset outperforms both the Dual-Use and the BAD+ datasets on a total of four different variants of the GPT-3.5 and Alpaca-LoRA model families [15]. A selection of the evaluation results measured by Deng et al. can be seen in Table 2. It is, however, important to notice that Deng et al. apply the same harmfulness score used to generate the SAP datasets as a measure of comparison to evaluate SAP against the other datasets. Since the compared datasets were created without taking this specific harmfulness score into consideration, an external metric could provide a more neutral evaluation.

Table 2: Harmfulness scores assigned to outputs from prompt injection attacks on different models and datasets as measured by Deng et al. The exact model identifiers of the GPT-3.5 models are `gpt3.5-turbo-0301` and `text-davinci-003` [15].

Dataset	Model			
	GPT-3.5		Alpaca-LoRA	
	turbo	davinci	7B	13B
Dual-Use	5.41	6.35	6.66	6.33
BAD+	0.63	1.87	4.12	3.44
SAP30	**8.70**	**7.18**	**8.80**	**8.72**

An important limitation of the automation framework proposed by Deng et al. is that it requires an initial attack prompt set as a baseline to start the LLM-based rephrasing process. If no suitable baseline dataset is available for a specific attack goal, a manual dataset has to be constructed [15]. This adds time-intensive and expensive human labor to the framework, preventing the semi-automated approach from being fully automated. In addition to the increasing effort, the baseline attack prompt dataset might also have an impact on the framework's performance. The effectiveness of the new attack prompts, which are created by the framework through LLM-based rephrasing, is determined by the type of model, the model's system prompt, and the provided in-context learning examples. While Deng et al. do not investigate the impact of using different baseline attack prompt datasets for the same attack task, the choice of baseline prompts likely impacts the framework's performance. For example, an LLM is unlikely to come up with completely new attack prompt strategies rather than rephrasing attack approaches that are included in the baseline attack prompts.

4.3 Supervised Fine-Tuning and RLHF

Combining the attack prompt effectiveness measurements from Perez and Ribeiro and Deng et al. allows us to draw some qualitative conclusions about the robustness of different fine-tuning approaches. As mentioned in Section 3.2, Perez and Ribeiro find that the `text-davinci-002` model from the GPT-3.5 series is significantly more susceptible to prompt injection attacks than the `text-davinci-001` model [38]. As pointed out in Section 3.2, `text-davinci-002` is likely not fine-tuned with a true RLHF approach.

Deng et al. make a similar observation in their comparison of `text-davinci-003` from the GPT-3.5 series with models from the Alpaca series. In contrast to `text-davinci-002`, `text-davinci-003` is fine-tuned with an RLHF approach as described by Ouyang et al. [24]. Based on the harmfulness scores from Table 2, they conclude that the GPT-3.5-based models are more robust against prompt injection attacks than the Alpaca models, which are fine-tuned using a self-instruct framework [51]. This conclusion aligns with the findings by Perez and Ribeiro and suggests that fine-tuning with RLHF leads to increased model robustness compared to classic supervised fine-tuning.

It is important to mention several aspects about this qualitative conclusion. First of all, the observation by Deng et al. that GPT-3.5 series models are more robust against prompt injection attacks than Alpaca models is based on the comparison of models that likely differ in size. While the parameter numbers of the Alpaca models used by Deng et al. are known, the parameter numbers of the GPT-3.5 series remain undisclosed. At least the `text-davinci-003` model is likely to have a much higher parameter number compared to the Alpaca models, which could impact robustness. Second, there is no data available to directly compare the findings by Perez and Ribeiro and Deng et al. The comparison is only based on the findings that in both works, RLHF fine-tuned models are less susceptible to prompt injection attacks than comparable models using alternative fine-tuning methods. Lastly, the comparison of RLHF and other supervised fine-tuning methods relies on the empirical comparison of existing models. The fact that models fine-tuned with RLHF are more robust does not mean that RLHF is, in principle, less susceptible to prompt injection attacks or that other fine-tuning methods are unable to produce sufficiently robust LLMs.

5 Gradient-Based Prompt Optimization

As mentioned in Section 3, Perez and Ribeiro achieve high attack success rates when carrying out a manual prompt injection attack against early GPT-3 LLMs in late 2022. However, the effectiveness of early approaches, like the one proposed by Perez and Ribeiro, decreases over time. This becomes evident, for example, when Hui et al. re-evaluate the attack approach of Perez and Ribeiro on more modern LLMs in late 2024. They find that the attack prompts of Perez and Ribeiro are unable to recover any part of a model's system prompt in the majority of the attacks [22].

The ineffectiveness of older prompt injection attacks on modern models is likely due to adjusted fine-tuning. Attack prompts from existing prompt injection attacks can be taken into account during the fine-tuning of new LLMs, which reduces the effectiveness of the attacks. Deng et al. prove that this works by fine-tuning models on their own attack prompts, which significantly reduces the effectiveness of the attacks [15]. Nevertheless, using existing prompt injection attacks to fine-tune new models does not make models fundamentally robust against attacks, which we further discuss in Section 6.1. However, it does drive the innovation of new and adapted attack approaches, such as gradient-based attack prompt optimization. In the following, we

give an overview of these more recent approaches by the example of attack frameworks proposed by Hui et al. and Shi et al. and compare them to the attack approaches discussed in Section 3 and Section 4.

5.1 Loss Minimization Approach

The approaches of both Hui et al. and Shi et al. follow the same basic principle to create attack prompts for prompt injection attacks. They define the attack prompt generation as an optimization problem, using a loss function $\mathcal{L}$ to quantify the deviation of the attacked model's actual response from the response targeted by the attack [22, 47]. For example, Shi et al. use the negative log-likelihood of the embedding vectors $\mathbf{r}_i$ that form the model's response to define the loss of an attack response. When the model is given the adversarial prompt $\tilde{\mathbf{p}}$, which was built with the injected instructions $\mathbf{s}^e$, the model response aimed for by the attack can be expressed as a series of embedding vectors $\tilde{\mathbf{r}^*} = \mathbf{r}_1^*, \mathbf{r}_2^*, ...\mathbf{r}_{n_r}^*$. Meanwhile the actual response $\tilde{\mathbf{r}}$, which might differ from the intended response $\tilde{\mathbf{r}*}$, can be equivalently expressed as $\tilde{\mathbf{r}} = \mathbf{r}_1, \mathbf{r}_2, ...\mathbf{r}_n$. The loss of the proposed attack prompt $\tilde{\mathbf{p}}$ can then be expressed as:

$$\mathcal{L}(\tilde{\mathbf{p}}) = -log \prod_{i=1}^{n_r} \Pr(\mathbf{r}_i^* | \tilde{\mathbf{p}}, \mathbf{r}_1, \mathbf{r}_2, ..., \mathbf{r}_{i-1}) \text{ [22]} \tag{5}$$

In other words, the loss is based on the probability that the model produces the attacker's desired output $\tilde{\mathbf{r}^*}$ when injected with the malicious instructions $\tilde{\mathbf{p}}$. The objective of the attack prompt construction is then to find optimal attack prompts that minimize the loss:

$$\underset{\tilde{\mathbf{p}}}{\text{argmin}}\ \mathcal{L}(\tilde{\mathbf{p}}) \tag{6}$$

and therefore maximize the probability that the model's response is the desired output $\tilde{\mathbf{r}^*}$ [22, 47].

To iteratively find a solution to the optimization problem, Hui et al. and Shi et al. use gradient-based approaches [22, 47]. This is done in a three-step process. In the first step, the responses of the model for a proposed attack prompt dataset are obtained by inputting the injected prompts into the model and collecting the corresponding output. In the second step, the loss of the model's responses is calculated according to Equation (5) while considering the attacker's desired responses for the corresponding attack. In the third step, the attack prompts are updated to minimize the calculated loss by approximating the loss of updated attack prompts. These three steps are carried out iteratively either for a certain number of iterations or until the algorithm converges. The resulting attack prompt dataset then minimizes the defined loss and therefore maximizes the desired model response in prompt injection attacks on systems using that model. The full iterative algorithm is conceptually depicted in Figure 3.

The main difficulty of this loss-based optimization approach is the discrete nature of the optimization problem. In continuous gradient-descent approaches, the gradient of the loss is used to update the optimized parameter directly, typically by subtracting the gradient from the current iteration's input [42]. However, not only the model's response $\tilde{\mathbf{r}}$, but also the compromised prompt $\tilde{\mathbf{p}} = \mathbf{p}_1, \mathbf{p}_2, ..., \mathbf{p}_{n_p}$ are series of embedding vectors. Therefore, updates to an attack prompt can only be made discretely by exchanging an embedding vector $\mathbf{p}_i$ with an alternative embedding vector $\mathbf{p}_i'$. This means that no continuous updates can be made

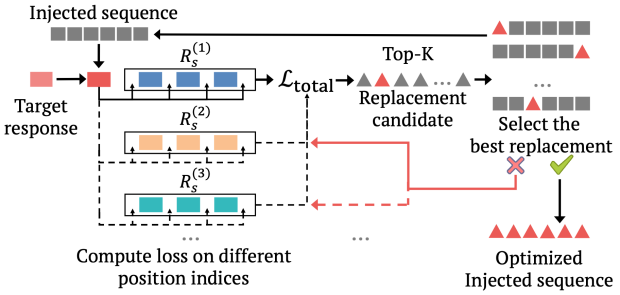


Figure 3: Iterative optimization of tokens in attack prompts based on a loss function [47].

to the attack prompt.

Both Hui et al. and Shi et al. solve this problem by estimating the impact of changing individual embedding vectors in the attack prompt [22, 47]. If an updated attack prompt $\tilde{\mathbf{p}}' = \mathbf{p}_1, ..., \mathbf{p}_i', ..., \mathbf{p}_{n_p}$ is created by exchanging the embedding vector $\mathbf{p}_i$ in $\tilde{\mathbf{p}}$ with $\mathbf{p}_i'$, the loss of the updated prompt can be expressed with Equation (5) as $\mathcal{L}(\tilde{\mathbf{p}}')$. The loss of this updated attack prompt can then be approximated with a first-order Taylor expansion as:

$$\mathcal{L}(\tilde{\mathbf{p}}') \approx \mathbf{p}_i' \ \nabla_{p_i} \mathcal{L}(\tilde{\mathbf{p}}) \text{ [22]}, \tag{7}$$

where ∇_{p_i} denotes the gradient with respect to the ith embedding vector in $\tilde{\mathbf{p}}$. Notably, the approximation does not contain $\mathcal{L}(\tilde{\mathbf{p}}')$, only $\mathcal{L}(\tilde{\mathbf{p}})$. This means that to evaluate the impact of a discrete update to the attack prompt, the updated prompt does not have to be fed through the model. Therefore, the approximation allows for finding an optimal new embedding vector for each position i in the attack prompt by searching through the set of discrete embedding vectors [22, 47].

Due to the presented approximation, it is not guaranteed that the update to the attack prompt that minimizes the approximated loss in Equation (7) also minimizes the true loss in Equation (5). Hui et al. and Shi et al. mitigate this effect in a similar way. They both find the k best embedding vector updates for each position i in the attack vector according to the approximated loss in Equation (7) [22, 47]. Hui et al. then compute the true loss for each of the k possible updates and find the optimal update amongst the k pre-computed options [22]. Shi et al. use a greedy search with random sampling on the k candidate updates instead [47].

5.2 Shadow Models

While the loss-defined optimization approach presented in Section 5.1 allows for strategically choosing optimal updates to the attack prompts based on the attacked model's response, it does not conform to the threat model introduced in Section 2.2. Since computing the gradient in Equation (7) requires access to the LLM's internal weights and calculations, it is not possible to perform this optimization when only black-box access to the attacked model is assumed.

The naive solution to this is to change the threat model to assume white-box model access instead of black-box access. However, this is not a realistic threat model for applications that use proprietary and closed-source LLMs. Considering the fact that for many of the most capable and widely used LLMs like GPT-5 [35], Gemini 3 [19], and Claude 4.5 [4], the internal weights are not

available, this assumption would render gradient-based prompt injection attacks ineffective in many scenarios. Both Hui et al. and Shi et al. opt for threat models that preserve the black-box access to the attacked application by using shadow models [22, 47].

Shadow models are used in the attack prompt optimization as a stand-in for the attacked model in the target application. While the attacker only has black-box access to the LLM used in the attacked application, white-box access can be assumed for the shadow model [22, 47]. Several powerful LLMs are available under open-weight licenses, which can be used as shadow models. Hui et al. and Shi et al. use, amongst others, models from the Llama [2] and Mistral [1] families, as well as GPT-J [49]. Therefore, this hybrid threat model of white-box access to the shadow model and black-box access to the target model can be considered to be as strong as the threat model described in Section 2.2. During the attack prompt optimization, the shadow model is used together with a shadow dataset consisting of system and user prompts that are assumed to be similar to the ones encountered in an attack on the actual application [22, 47]. Whether actual data from the system can be collected by the attacker or whether assumptions have to be made to construct the shadow dataset depends on the additional attacker capabilities described in Section 2.2.

Whether prompt injection attacks using shadow models are effective against the targeted application mainly depends on the transferability of the attack from the shadow model to the LLM used in the application. Both Hui et al. and Shi et al. find that attacks trained on a shadow model that differs from the model used by the targeted application transfer with high effectiveness [22, 47]. While attack prompt effectiveness does decrease in some cases, especially when transferring an attack from a smaller shadow model to a larger target model, the attacks remain successful in most cases [22, 47]. Considering that an attacker might know which model is used by the targeted application, a suitable shadow model with proven high transferability and similar size can be chosen as the shadow model. Hui et al. also prove that attacks can remain highly effective even when the shadow dataset used in the attack prompt optimization significantly differs from data found in the target application [22].

5.3 Comparison to LLM-based Rephrasing

The main advantage of the newer gradient-based approaches over the older approaches presented in Section 3 and Section 4 is their higher attack effectiveness on modern LLMs. Hui et al. find that in a prompt leaking scenario, their approach can recover an exact system prompt in up to 68% of the attacks on modern LLMs, including `LLaMA-2-7B` [22]. Compared to that, the attacks developed by Perez and Ribeiro only recover the exact system prompt in 2% of the attacks [22]. Similarly, Shi et al. achieve an average attack success rate of 97% on the `Llama-3-8B-Instruct` model [47]. Both Hui et al. and Shi et al. conclude that their newer gradient-based approaches are significantly more effective than previous prompt injection approaches on modern LLMs [22, 47].

The superior attack effectiveness of gradient-based methods can be attributed to their more direct optimization approach. Manual or automated rephrasing approaches like the ones by Perez and Ribeiro and Deng et al. adjust attack prompts on a word level and base the updates made to attack prompts on general prompting rules and guidelines. In contrast to that, gradient-based approaches like those by Hui et al. and Shi et al. adjust attack prompts on a token level and select attack prompt adjustments based directly on the attacked model's responses. It can therefore be concluded that gradient-based methods are conceptually better suited to create attack prompts optimized for prompt injection attacks. The better optimization does, however, come at the cost of the need for training data and a more complex attack prompt creation process.

Another advantage of gradient-based methods over rephrasing methods is that they are less dependent on the initial attack prompts used as a starting point for the optimization process. Hui et al. even find that an initialization with random embedding vectors is the best-performing initialization for their approach [22]. Since the replacement of tokens in the attack prompt is chosen by searching through the token alphabet and not via in-context learning generation, gradient-based approaches are more likely to find optimized attack prompts that significantly differ from the initial prompts. On the other hand, gradient-based approaches are dependent on a good transferability of the attack prompts from the shadow model they were optimized on to the model in the attacked application. Since rephrasing-based approaches work without a shadow model, they tune the attack prompts directly on the attacked LLM.

The adjustment of gradient-based attacks to different attack goals is done by changing the loss function used for the optimization. This not only allows for different attack goals but also enables additional specialization. For example, Shi et al. use three loss terms balanced by hyperparameters [47]. One loss term aims at increasing the likelihood that the attack produces the desired response, similar to the loss defined in Equation (5). The other two terms are introduced to make the attack robust against variations that are encountered in the attack scenario considered by Shi et al. and to lower the attack prompt's perplexity to evade attack detection. Introducing additional loss objectives and balancing the relative importance of loss terms allows for simple but effective specialization and adaption of gradient-based prompt injection attacks.

5.4 Realistic Attack Scenarios

Another novelty of the prompt injection attacks proposed by Hui et al. and Shi et al. in comparison to older attacks like the ones by Perez and Ribeiro and Deng et al. is that they consider more realistic attack scenarios. Hui et al. do not only evaluate their attacks on offline applications but also test the effectiveness of their attack against real-world LLM applications on the Poe platform [22]. For a prompt leaking attack against offline applications, they find an average success rate for recovering the exact system prompt is 82.3%. While the average exact match on real-world applications is lower at 68.0%, it proves that their attack is suitable not only for attacks on experimental setups [22]. Shi et al. motivate their attack against LLM-as-a-judge applications with possible real-world targets like search result ranking, LLM tool selection frameworks, and reinforcement learning from AI feedback approaches [47]. All of these areas are already using or will likely adopt LLM-as-a-judge-based evaluation and ranking and are therefore an attractive target for prompt injection attacks similar to the one proposed by Shi et al. In case studies to evaluate the efficacy of their attack against real-world applications in all three

areas, Shi et al. find that their attack is effective in all of them [47].

Besides the more realistic attack targets, both Hui et al. and Shi et al. also consider the possibility of defenses against prompt injection attacks in their proposed attacks. Hui et al. use so called adversarial transformations to make their prompt leaking attack robust against output filters in the attacked applications. In such a defense, the application searches for exact or partial matches of the system prompt in the LLM's output [22]. Hui et al. adjust their attack prompts to perform reversible transformations like prefixes to every sentence in order to evade possible output filters and then reverse the transformations on the attacker end [22]. Shi et al. use an additional balanced loss term to make a low perplexity of the attack prompts one of the optimization targets. This approach can help to evade defenses based on the fact that optimized adversarial prompts often have higher perplexity than typical user prompts [47].

6 Challenges of Instruction-Data Integration

While older prompt injection attacks become less effective on newer models, a consistent theme of prompt injection research is that the most recent attack approaches are highly effective even against state-of-the-art models. In this section, we discuss defense approaches against prompt injection attacks, based on the fundamental challenge presented by the close integration of data and instructions in LLM inputs. The separation of data and instructions is one of the fundamental principles of cybersecurity, which is typically applied by in-memory separation of non-executable memory for potentially unsafe data inputs [37]. For LLMs, a fundamental separation of data and instructions is not possible because they are designed architecturally to handle all inputs, including instructions, identically [27].

The inseparability of data and instructions forms the basis of all prompt injection attacks and poses a significant challenge for designing effective defense strategies. Since jailbreaking attacks against LLMs are based on the same fundamental principle of instruction-data integration, we are also considering defense approaches against these attacks in this chapter without further separation between the two attack classes. In the following, we present three main classes of defense approaches. Starting with the currently widely adopted approaches, each class has higher potential for long-term robustness, but at the same time, is harder to apply effectively at scale.

6.1 Current Defense Approaches

The majority of defense approaches in use today are based on the analysis of existing prompt injection strategies. Some of the most prominent examples are the sanitation of inputs based on known attack patterns [44], filters looking for injected behavior like leaked system prompts or harmful speech in model outputs [22, 26], monitoring LLM behavior and function usage [44], and the sandboxing of systems to limit the effects of successful attacks [14]. All of these approaches have the advantage of being applicable after the LLM's training phase and can be adapted to new defense strategies without having to adjust model parameters.

Another approach is to harden an LLM against prompt injection during fine-tuning [12, 15]. While it is a higher computational effort due to the need to adjust model parameters, fine-tuning has the advantage of being an integrated defense mechanism without additional processing steps. Additionally, it has the potential to generalize to new attacks to some extent without the need of adapting the defense implementation. Deng et al. prove both the effectiveness and generalizability of this approach by hardening two sizes of an Alpaca model. They do so by fine-tuning the models to produce refusal responses to three of their own SAP attack prompt datasets, which we presented in Section 4.2 [15]. The harmfulness scores measured in this experiment are shown in Table 3. Deng et al. find that the output harmfulness of both model sizes attacked with the SAP20 dataset is significantly reduced when fine-tuned on other SAP datasets. While the harmfulness scores are also reduced when attacked with datasets that are not from the SAP family, the effect is significantly lower [15]. This shows that the generalizability of fine-tuning defenses based on existing attacks is limited.

Table 3: Harmfulness scores of Alpaca 7B and 13B model variants before and after fine-tuning. The rows correspond to the models before fine-tuning and after being fine-tuned on the SAP5, SAP10, or SAP30 dataset. The columns show the resulting harmfulness scores on the related SAP20 and the non-related Dual-Use and BAD+ datasets [15].

		Dataset		
#Params	**Fine-tune**	SAP20	Dual-Use	BAD+
7B	-	8.49	6.63	4.12
	SAP5	0.01	2.14	1.69
	SAP10	0.06	2.08	0.96
	SAP30	0.01	1.96	0.93
13B	-	8.30	6.33	3.44
	SAP5	1.07	3.39	1.83
	SAP10	0.25	1.06	1.25
	SAP30	2.97	4.57	1.53

Defense approaches like the ones mentioned above are widely adopted in modern LLM-based applications and form the basis of the defense strategies stated by companies like OpenAI [34] and NVIDIA [5]. The problem shared amongst all these defense strategies is that they are based on existing prompt injection attacks. Therefore, novel attack strategies that are sufficiently distinct from existing ones can effectively attack models despite the implemented defenses. Since, in most realistic threat models, any employed defense mechanisms are assumed to be known by the attacker, new attacks can be designed specifically to circumvent existing defenses. Examples of this are the specialized loss terms for gradient-based methods as used by Shi et al. in Section 5.4 [47] or manual approaches like the recent example of adversarial poetry [9]. While this makes this class of defense strategies useful as short-term fixes for prompt injection attacks, they will remain band-aid solutions due to their reactive nature.

6.2 Separating Instructions and Data

While the fundamental architecture of LLMs does not allow for fully separating instructions and data, there are several approaches that try to approximate this paradigm. One approach is to separate the computation that an LLM performs on instructions and data. This can be achieved, for example, by architectural design patterns specifically adopted for LLM-integrated

applications [7] or through a direct separation of data flows for trusted instructions and untrusted data inputs [14]. Another approach is to make instructions and data more distinguishable for LLMs. The high effectiveness of prompt injection attacks can be at least partially attributed to the inability of LLMs to distinguish between trustworthy instructions and untrustworthy data inputs [55]. Making the embeddings of the two input types more distinguishable for the model can help to increase robustness [56].

The fact that these approaches are based less directly on existing prompt injection attacks makes them more robust against novel attack strategies. While they are, in theory, applicable at scale, implementing them takes more careful consideration of model defense during the design and training phases of LLM-based applications. They are therefore more robust, but less easily adoptable than the approaches presented in Section 6.1. Therefore, these approaches can be used as a step-up in model robustness, as the potential severity of successful prompt injection attacks increases, as discussed in Section 1.

6.3 Defending with Interpretability

The most promising class of defense approaches is techniques based on mechanistic interpretability. Mechanistic interpretability aims to achieve deep causal insight into the input processing of LLMs through reverse-engineering techniques. In contrast to other approaches that treat LLMs as black-boxes, mechanistic interpretability techniques use information on the structure and internal processing of models for a granular understanding of model behavior [40, 43]. Compared to black-box methods, interpretability methods are concerned with answering the question of '*How* does the model work internally?' rather than '*Why* was this output produced for this input?' [42]. Two main classes of mechanistic interpretability techniques are attributing functions to internal model components and finding vector-represented concepts within the model's internal activations [40].

Currently, there are several attempts to understand attack causality [18, 29, 30], proposals for defenses [6], as well as attack techniques based on the interpretability of attacks on LLMs [53]. Using mechanistic interpretability techniques to analyze adversarial attacks on LLMs has the potential of providing a deeper and more causal understanding of why attacks are successful. This knowledge can then be used to implement defenses based on the fundamental nature of prompt injection attacks. Therefore, defenses based on interpretability have the potential to be much more generalizable than existing defenses and could lead to highly improved model robustness.

On the other hand, there are several issues with interpretability-based techniques. The scientific field of mechanistic LLM interpretability, while quickly evolving, is overall in early development [40, 43]. One of the main problems is that most mechanistic interpretability approaches are limited to toy examples, small models, and a very limited scope [40]. This means that scaling interpretability-based defenses to apply to state-of-the-art LLMs presents a significant challenge but represents one of the best approaches to limiting prompt injection attack success in the future.

7 Conclusion

We gave an overview of prompt injection attack techniques against LLMs, beginning with the formal definition of prompt injection and the basic threat model shared amongst attack approaches. Besides the temporal development process of prompt injection approaches, we structured the techniques into two distinct classes. One consisting of word-level attack prompt construction and the other containing token-level gradient-based attack prompt optimization.

Following the development of prompt injection attacks, we presented a manual approach to creating attack prompts based on general prompting guidelines. We then examined the next development step by discussing a semi-automated approach. By comparing the data of several case studies, we concluded that better instruction following, enabled by larger models and better fine-tuning, makes LLMs more susceptible to prompt injection attacks.

We proceeded to present the second class of gradient-based approaches, due to the observation that older attack techniques become less effective against newer LLMs. By comparing gradient-based approaches to LLM-based attack prompt rephrasing, we highlighted that gradient optimization has the advantages of high attack effectiveness and easy specialization via multiple loss terms. We also pointed out the higher complexity of gradient-based approaches, which is due to their need for a shadow model and higher computational effort. In reference to the development progress of prompt injection attacks, we noted the more realistic attack scenarios discussed in newer studies.

Lastly, we discussed the fundamental problem of instruction-data integration that enables the ongoing high success rates of prompt injection attacks on state-of-the-art LLMs. We presented three types of defense strategies. The band-aid solutions that are currently widely adopted, deeper architectural adjustments with potentially better generalizability, and interpretability-based defenses that could offer a conceptual solution to prompt injection attacks if they become applicable at scale.

Overall, prompt injection attacks against LLMs remain one of the most dominant threats to LLM-based applications. Considering the increasing adoption and impact of such applications, novel defense mechanisms like interpretability-based defenses are needed to underlay further efforts of LLM-based automation.

References

[1] Mistral AI. 2023. Mistral-7B-Instruct-v0.2. https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2

[2] AI@Meta. 2024. Meta-Llama-3-8B-Instruct. https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct

[3] Yadagiri Annepaka and Partha Pakray. 2025. Large language models: a survey of their development, capabilities, and applications. *Knowledge and Information Systems* 67, 3 (March 2025), 2967–3022. doi:10.1007/s10115-024-02310-4

[4] Anthropic. 2025. Claude Sonnet 4.5. https://www.anthropic.com/claude/sonnet

[5] atefeh.asayesh. 2023. Securing LLM Systems Against Prompt Injection. https://developer.nvidia.com/blog/securing-llm-systems-against-prompt-injection/

[6] Leonard Bereska. 2024. Mechanistic Interpretability for Adversarial Robustness — A Proposal. *Self-published* (Aug. 2024). https://leonardbereska.github.io/blog/2024/mechrobustproposal

[7] Luca Beurer-Kellner, Beat Buesser, Ana-Maria Creţu, Edoardo Debenedetti, Daniel Dobos, Daniel Fabian, Marc Fischer, David Froelicher, Kathrin Grosse, Daniel Naeff, Ezinwanne Ozoani, Andrew Paverd, Florian Tramèr, and Václav Volhejn. 2025. Design Patterns for Securing LLM Agents against Prompt Injections. (2025). doi:10.48550/ARXIV.2506.08837

[8] G. Bharathi Mohan, R. Prasanna Kumar, P. Vishal Krishh, A. Keerthinathan, G. Lavanya, Meka Kavya Uma Meghana, Sheba Sulthana, and Srinath Doss. 2024. An analysis of large language models: their impact and potential applications. *Knowledge and Information Systems* 66, 9 (Sept. 2024), 5047–5070. doi:10.1007/s10115-024-02120-8

[9] Piercosma Bisconti, Matteo Prandi, Federico Pierucci, Francesco Giarrusso, Marcantonio Bracale, Marcello Galisai, Vincenzo Suriani, Olga Sorokoletova, Federico Sartore, and Daniele Nardi. 2025. Adversarial Poetry as a Universal Single-Turn Jailbreak Mechanism in Large Language Models. doi:10.48550/arXiv.2511.15304

[10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[11] Shreyas Chaudhari, Pranjal Aggarwal, Vishvak Murahari, Tanmay Rajpurohit, Ashwin Kalyan, Karthik Narasimhan, Ameet Deshpande, and Bruno Castro da Silva. 2025. RLHF Deciphered: A Critical Analysis of Reinforcement Learning from Human Feedback for LLMs. *ACM Comput. Surv.* 58, 2 (Sept. 2025), 53:1–53:37. doi:10.1145/3743127

[12] Sizhe Chen, Julien Piet, Chawin Sitawarin, David Wagner, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri Chaudhuri, and Chuan Guo. 2025. Defending against Prompt Injection with Structured Queries (StruQ) and Preference Optimization (SecAlign). http://bair.berkeley.edu/blog/2025/04/11/prompt-injection-defense/

[13] Zina Chkirbene, Ridha Hamila, Ala Gouissem, and Unal Devrim. 2024. Large Language Models (LLM) in Industry: A Survey of Applications, Challenges, and Trends. In *2024 IEEE 21st International Conference on Smart Communities: Improving Quality of Life using AI, Robotics and IoT (HONET)*. 229–234. doi:10.1109/HONET63146.2024.10822885 ISSN: 1949-4106.

[14] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. (2025). doi:10.48550/ARXIV.2503.18813

[15] Boyi Deng, Wenjie Wang, Fuli Feng, Yang Deng, Qifan Wang, and Xiangnan He. 2023. Attack Prompt Generation for Red Teaming and Defending Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 2176–2189. doi:10.18653/v1/2023.findings-emnlp.143

[16] OWASPGenAIProject Editor. [n. d.]. LLM07:2025 System Prompt Leakage. https://genai.owasp.org/llmrisk/llm07-insecure-plugin-design/

[17] Leo (EleutherAI) Gao. 2021. On the Sizes of OpenAI API Models. https://blog.eleuther.ai/gpt3-model-sizes/

[18] Jorge García-Carrasco, Alejandro Maté, and Juan Trujillo. 2024. Detecting and Understanding Vulnerabilities in Language Models via Mechanistic Interpretability, Vol. 1. 385–393. doi:10.24963/ijcai.2024/43 ISSN: 1045-0823.

[19] Google. 2025. A new era of intelligence with Gemini 3. https://blog.google/products-and-platforms/products/gemini/gemini-3/

[20] Yanchu Guan, Dong Wang, Zhixuan Chu, Shiyu Wang, Feiyue Ni, Ruihua Song, and Chenyi Zhuang. 2024. Intelligent Agents with LLM-based Process Automation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 5018–5027. doi:10.1145/3637528.3671646

[21] Krystal Hu and Krystal Hu. 2023. ChatGPT sets record for fastest-growing user base - analyst note. *Reuters* (Feb. 2023). https://www.reuters.com/technology/chatgpt-sets-record-fastest-growing-user-base-analyst-note-2023-02-01/

[22] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. PLeak: Prompt Leaking Attacks against Large Language Model Applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*. Association for Computing Machinery, New York, NY, USA, 3600–3614. doi:10.1145/3658644.3670370

[23] Jan Leike [@janleike]. 2022. Discussion on OpenAI model relation to RLHF paper. https://x.com/janleike/status/1584618242756132864

[24] janus. 2022. Update to Mysteries of mode collapse: text-davinci-002 not RLHF. (Nov. 2022). https://www.alignmentforum.org/posts/mbGjzyy6eJXT4gFpm/update-to-mysteries-of-mode-collapse-text-davinci-002-not

[25] Hyeongyo Jeong, Haechan Lee, Changwon Kim, Sungtae Shin, Hyeongyo Jeong, Haechan Lee, Changwon Kim, and Sungtae Shin. 2024. A Survey of Robot Intelligence with Large Language Models. *Applied Sciences* 14, 19 (Oct. 2024). doi:10.3390/app14198868

[26] Jessica Ji, Josh Goldstein, and Andrew Lohn. 2023. Controlling Large Language Model Outputs: A Primer. https://cset.georgetown.edu/publication/controlling-large-language-models-a-primer/

[27] Uday Kamath, Kevin Keenan, Garrett Somers, and Sarah Sorenson. 2024. Prompt-based Learning. In *Large Language Models: A Deep Dive: Bridging Theory and Practice*. Springer Nature Switzerland, Cham, 83–133. doi:10.1007/978-3-031-65647-7_3

[28] Yeseung Kim, Dohyun Kim, Jieun Choi, Jisang Park, Nayoung Oh, and Daehyung Park. 2024. A survey on integration of large language models with intelligent robots. *Intelligent Service Robotics* 17, 5 (Sept. 2024), 1091–1107. doi:10.1007/s11370-024-00550-5

[29] Nathalie Maria Kirch, Constantin Niko Weisser, Severin Field, Helen Yannakoudakis, and Stephen Casper. 2025. What Features in Prompts Jailbreak LLMs? Investigating the Mechanisms Behind Attacks. In *Proceedings of the 8th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, Yonatan Belinkov, Aaron Mueller, Najoung Kim, Hosein Mohebbi, Hanjie Chen, Dana Arad, and Gabriele Sarti (Eds.). Association for Computational Linguistics, Suzhou, China, 480–520. doi:10.18653/v1/2025.blackboxnlp-1.28

[30] Yuping Lin, Pengfei He, Han Xu, Yue Xing, Makoto Yamada, Hui Liu, and Jiliang Tang. 2024. Towards Understanding Jailbreak Attacks in LLMs: A Representation Space Analysis. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 7067–7085. doi:10.18653/v1/2024.emnlp-main.401

[31] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. doi:10.48550/arXiv.2310.12815

[32] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. 2025. Large Language Model Agent: A Survey on Methodology, Applications and Challenges. (2025). doi:10.48550/ARXIV.2503.21460

[33] John McDonnell. 2022. OpenAI comes clean about GPT 3.5. https://jmcdonnell.substack.com/p/openai-comes-clean-about-gpt-35

[34] OpenAI. [n. d.]. Understanding prompt injections. https://openai.com/safety/prompt-injections/

[35] OpenAI. 2025. Introducing GPT-5. https://openai.com/index/introducing-gpt-5/

[36] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems* 35 (Dec. 2022), 27730–27744. https://papers.nips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html

[37] Open Worldwide Application Security Project (OWASP). 2025. A05 Injection - OWASP Top 10:2025. https://owasp.org/Top10/2025/A05_2025-Injection/

[38] Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques For Language Models. https://openreview.net/forum?id=qiaRo_7Zmug

[39] preamble. 2022. Discovery and Disclosure of Prompt Injections. https://www.preamble.com/prompt-injection-a-critical-vulnerability-in-the-gpt-3-transformer-and-how-we-can-begin-to-solve-it

[40] Daking Rai, Yilun Zhou, Shi Feng, Abulhair Saparov, and Ziyu Yao. 2024. A Practical Review of Mechanistic Interpretability for Transformer-Based Language Models. (2024). doi:10.48550/ARXIV.2407.02646

[41] Mubashar Raza, Zarmina Jahangir, Muhammad Bilal Riaz, Muhammad Jasim Saeed, and Muhammad Awais Sattar. 2025. Industrial applications of large language models. *Scientific Reports* 15, 1 (April 2025), 13755. doi:10.1038/s41598-025-98483-1 Publisher: Nature Publishing Group.

[42] Stuart J. Russell and Peter Norvig. 2021. *Artificial intelligence: a modern approach* (fourth edition ed.). Pearson, Hoboken, NJ.

[43] Naomi Saphra and Sarah Wiegreffe. 2024. Mechanistic? https://openreview.net/forum?id=schAf4BPtD

[44] Purushottam Sarsekar and Shezan Rohinton Mirzan. [n. d.]. Prompt Injection | OWASP Foundation. https://owasp.org/www-community/attacks/PromptInjection

[45] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. 2025. The Prompt Report: A Systematic Survey of Prompt Engineering Techniques. doi:10.48550/arXiv.2406.06608

[46] Sarama Shehmir, Rasha Kashef, Sarama Shehmir, and Rasha Kashef. 2025. LLM4Rec: A Comprehensive Survey on the Integration of Large Language Models in Recommender Systems—Approaches, Applications and Challenges. *Future Internet* 17, 6 (June 2025). doi:10.3390/fi17060252

[47] Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. 2024. Optimization-based Prompt Injection Attack to LLM-as-a-Judge. *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Dec. 2024), 660–674. doi:10.1145/3658644.3690291

[48] Chunliang Tao, Xiaojing Fan, and Yahe Yang. 2024. Harnessing LLMs for API Interactions: A Framework for Classification and Synthetic Data Generation. In *2024 5th International Conference on Computers and Artificial Intelligence Technology (CAIT)*. 628–634. doi:10.1109/CAIT64506.2024.10962957

[49] Ben Wang and Aran Komatsuzaki. 2021. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. https://huggingface.co/EleutherAI/gpt-j-6b

[50] Jiaqi Wang, Enze Shi, Huawen Hu, Chong Ma, Yiheng Liu, Xuhui Wang, Yincheng Yao, Xuan Liu, Bao Ge, and Shu Zhang. 2025. Large language models for robotics: Opportunities, challenges, and perspectives. *Journal of Automation and Intelligence* 4, 1 (March 2025), 52–64. doi:10.1016/j.jai.2024.12.003

[51] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki (Eds.). Association for Computational Linguistics, Toronto, Canada, 13484–13508. doi:10.18653/v1/2023.acl-long.754

[52] Simon Willison. 2022. Prompt injection attacks against GPT-3. https://simonwillison.net/2022/Sep/12/prompt-injection/

[53] Thomas Winninger, Boussad Addad, and Katarzyna Kapusta. 2025. Using Mechanistic Interpretability to Craft Adversarial Attacks against Large Language Models. doi:10.48550/arXiv.2503.06269

[54] Chaoyun Zhang, Shilin He, Jiaxu Qian, Bowen Li, Liqun Li, Si Qin, Yu Kang, Minghua Ma, Guyue Liu, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2025. Large Language Model-Brained GUI Agents: A Survey. *Transactions on Machine Learning Research* (Feb. 2025). https://openreview.net/forum?id=xChvYjvXTp

[55] Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert. 2024. Can LLMs Separate Instructions From Data? And What Do We Even Mean By That? https://openreview.net/forum?id=8EtSBX41mt

[56] Egor Zverev, Evgenii Kortukov, Alexander Panfilov, Soroush Tabesh, Sebastian Lapuschkin, Wojciech Samek, and Christoph H. Lampert. 2025. ASIDE: Architectural Separation of Instructions and Data in Language Models. https://openreview.net/forum?id=GlmqRQsCaI

Received 12 January 2026; revised ; accepted